

NDNSF: A Data-Centric Service Framework for Secure, Collaborative, and Dynamic Networked Applications

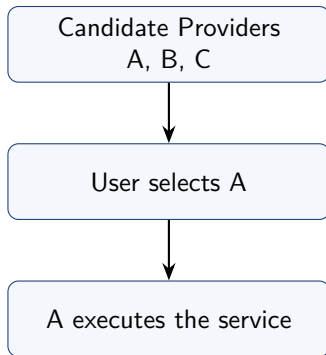
Tianxing Ma

Department of Computer Science
University of Memphis
Advisor: Dr. Lan Wang

PhD Proposal Defense
September 2026

Service Discovery: Several Candidates, One Executor

- ▶ The User knows the required service.
- ▶ Reachability, capability and willingness may change.
- ▶ Providers evaluate valid Requests and return positive or negative ACK Data.
- ▶ The User selects one eligible positive-ACK Provider.



Initial NDNSF: discover alternatives and select one Provider per invocation, not multiple collaborating roles. Arrows show the selection process.

From Invocation to Collaboration: Two Research Scopes

Foundation: evaluate discovery and authorization

The initial prototype selects one Provider. Discovery confidentiality, service permissions and management costs matter even without collaboration.

Extension: coordinate complementary roles

Some tasks need several Providers with different responsibilities. Separate calls do not define a shared role assignment or enforce dependencies between them.

Evaluate the invocation foundation; develop and validate the collaboration extension. Existing invocation results do not establish collaboration correctness.

NDN Primitives and Service-Level Decisions

NDN building block	Remaining application responsibility
Named Data and signatures [1]	Names support identity and forwarding; a Trust Schema [16] constrains signing keys
Interest retrieval and caching [1,26]	Check task identity, dependencies and deadlines; caches do not guarantee storage
Dataset synchronization / SVS [2,3]	Discover names; define service-message transitions
NAC [4] / enhanced NAC-ABE [19]	Encrypt Content for read access; check invocation permission separately

NAC: Name-based Access Control; ABE: attribute-based encryption; SVS: State Vector Sync.

NDN supplies data-centric communication. NDNSF defines how validated service messages authorize and advance an invocation.

Application Needs Determine the Research Scope

Application need	Framework question
Find a camera covering an area, or compute capacity for a model	Who is permitted, reachable and willing now?
Analyze images from several cameras, or execute dependent model fragments	Which Provider performs each role, and which inputs may it use?
Obtain useful output before a deadline, despite loss or outages	What completes, what fails, and what does recovery cost?

These needs lead to RQ1–RQ3. Invocation [9,10] and command authorization [12] provide precedents. This is a bounded framework scope, not a claim to cover every application requirement.

Three Research Questions

- ▶ **RQ1:** How can NDNFSF discover willing Providers, keep descriptors confidential from unauthorized readers, and check service permissions before execution, and at what cost?
- ▶ **RQ2:** How can single-Provider invocation be extended to roles and named dependencies across Providers, whether Data comes from producers or caches?
- ▶ **RQ3:** When do these mechanisms improve completion, latency, or execution efficiency, and when do their costs outweigh the benefits?

RQ1 evaluates the invocation foundation; RQ2 studies its collaboration extension; RQ3 measures both scopes separately.

Related Work and the Remaining Questions

Prior work	What the comparison must acknowledge
NFN [6], NFaaS [7]	Named computation and function execution/placement
RICE [9], NSC [10]	Invocation mechanisms; NSC checks signed calls using a Trust Schema
DNMP [12]	Trust Schema constrains roles and measurement commands
CaSCON [15]	Service-composition research; requires a feature-level comparison before claiming a new composition mechanism
gRPC [17,27]	Complete configurations can include discovery/resolver integration, not only fixed addresses

Collaboration research: make runtime assignments constrain intermediate Data use.
Discovery and authorization are evaluated separately.

Research Contributions and Current Evidence

Intended contribution	Current evidence boundary
Discovery / authorization design and tradeoffs	Invocation implementation and bounded experiments exist; full alternative lifecycle-cost comparison remains open
ACK-derived plan and local input checks	Design and implementation work; direct/cache and failure validation remains incomplete
Cross-case empirical findings	Existing mobility/work-efficiency and small-model results support limited configurations, not all new collaboration claims

Proposal status, September 2026: implemented, locally tested, measured, and fully qualified are not interchangeable. [28]

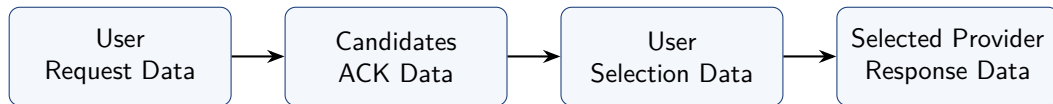
Participants and Trust Bootstrap

- ▶ User: requests a service and selects a Provider; the extension also assigns collaboration roles.
- ▶ Provider: offers and executes a service. A node may play both roles.
- ▶ Controller: administers identities and service permissions approved by the organization.
- ▶ Participants use provisioned trust anchors and validation rules for certificates and Controller material.

Unknown runtime candidates are not unregistered authorization identities. The User discovers current participation; the Controller does not choose this task's Providers.

Invocation Overview: Request a Timely Aerial Image

An operator requests an aerial image. Authorized camera Providers A and B consider the task.



- ▶ A returns a positive ACK; B is busy and returns a negative ACK.
- ▶ The User selects A; A checks Selection before capturing the image.

Arrows show application order. Data is retrieved with Interests; neither an ACK nor a network delivery alone authorizes execution.

Single-Provider Invocation: Four Named Data Types

Data	Logical purpose
Request	Describe required service and participation constraints; include protected discovery information and RequestNonce
ACK	Report positive or negative participation; positive ACK supplies selection information and ProviderChallenge
Selection	Assign selected execution; provide input/references and protected request-scoped keys; constructed using ProviderChallenge
Response	Return status, results or result references; the User still validates them before use

Sync helps discover names; Interests retrieve Data. The Provider checks Selection before execution; a positive ACK alone does not start the task.

Request: Discovery Descriptor; Selection: Task Input

Discovery descriptor

Needed to decide whether to participate: area, deadline, sensor/model and resource requirements.

Application input

Needed to execute the selected task: images, detailed processing parameters, or protected input references. Input and results use request-scoped symmetric keys. Selection delivers key material encrypted for the selected Provider; the requesting User retains the keys.

The boundary follows necessity. A location needed to assess coverage belongs in the protected descriptor; an image needed only for processing can wait.

Three Security Requirements; One Additional Benefit

1. Keep discovery descriptors confidential from unauthorized readers before selecting a Provider.
2. Authenticate participants, verify service permissions, and reject replayed invocation messages.
3. Grant and withdraw service permissions during operation.

Additional benefit: aggregate service permissions in a decryption key (DKEY) while retaining identity-signing credentials. This is a management property, not another security guarantee.

Reason 1: Confidential Discovery Before Selection

- ▶ The User knows the service, but not yet the currently willing Provider identities.
- ▶ An observation area and deadline can reveal intended activity even without the full application input.
- ▶ The User encrypts the descriptor using ABE and the service attribute, without listing Provider identities. [19]

Confidential from parties lacking the required decryption capability. Authorized but unselected Providers can read it; names and traffic patterns remain visible.

Reason 2: Reuse Service-Permission Management

- ▶ ABE was introduced to keep discovery descriptors non-public. [19]
- ▶ NDNFSF reuses the attribute authority to manage Provider and User service permissions. [28]
- ▶ RequestNonce and ProviderChallenge checks use these permissions during invocation.

ABE discovery plus role-based invocation is also possible. NDNFSF avoids adding role signing credentials solely for service membership; signature validation and request-state checks remain necessary. User DKEYs and challenge processing still add cost.

How NDNSF Checks Service Permissions

1. Provider permission: recover RequestNonce from ABE-protected Request content and return it in ACK Data. The User compares it with RequestNonce stored for this request.
2. User permission: recover ProviderChallenge from ABE-protected ACK content and construct Selection Data. The Provider validates Selection using its stored ProviderChallenge.

Also check current service permissions. These checks show access to protected values, not exclusive key possession. RequestNonce is not the Interest Nonce.

Identity and Replay Checks Have Different Jobs

Identity authentication

Validate Data signatures and certificate chains under trust rules. An accepted signing identity alone does not establish service permission.

Replay protection

Match messages to an active request and its state. Record whether ProviderChallenge has already been used to accept Selection for execution; duplicate Selection must not start another execution.

Retain or safely invalidate request state, including across restart. Encryption or a challenge value alone does not prevent replay.

Additional Benefit: Aggregate Service Permissions

- ▶ Within one Controller domain, combine several service permissions in one entity's DKEY; retain identity-signing credentials. [19,28]
- ▶ DNMP represents command authority through role keys and schema rules. One role can permit several commands. [12]
- ▶ Where equivalent permissions require distinct role credentials, NDNFSF avoids maintaining those extra signing credentials.

Fewer independently managed credentials does not imply fixed DKEY size or lower total cost. Policy size, key distribution and updates still matter.

DNMP: Trust-Schema Command Authorization

Distributed Network Measurement Protocol (DNMP) [12] authorizes measurement commands:

1. The Network Key anchors role keys, device keys, and the schema.
2. The client uses target, probe type, arguments, and user role to find an allowed signing key; among matching keys, it chooses the least privileged.
3. The receiving Network Observer Daemon validates the publication under the schema before dispatching the probe.

DNMP also uses publication lifetimes and retained state against replay, assuming loosely synchronized clocks. These are concrete command-authorization mechanisms, not just identity checks.

DNMP-inspired Authorization versus NDNFSF

Design	Execution authorization	Discovery confidentiality
DNMP-inspired [12]	Check role-constrained signatures and NDNFSF invocation state	Pair with service-group encryption or ABE
NDNFSF ABE-backed	Check signatures, RequestNonce, ProviderChallenge, current permissions and request state	User encrypts descriptor content with service-level ABE

Proposed comparison: retain NDNFSF discovery and Selection, with input/result protection and revocation checks in both designs. Trust Schema [16] alone does not encrypt Content; adapting DNMP authorization is not an original DNMP feature. No measured cost advantage is claimed.

When Another Authorization Design May Be Better

- ▶ Known Providers and simple rules: signed invocation plus recipient encryption may be sufficient.
- ▶ A small, stable receiver group: ordinary group-key management may be attractive.
- ▶ Rich command-argument restrictions: DNMP-style rules make those constraints explicit. [12]
- ▶ Many changing entitlements: measure both policy/key updates and credential/rule updates before claiming a management advantage.

Compare complete combinations of authorization and encryption. ABE-backed authorization is one design choice, not a universally superior solution.

Runtime Permission Grants and Withdrawals

- ▶ Both operations update Controller-signed permission status. [28]
- ▶ Ordinary grant: update the target's complete policy; it retrieves a replacement DKEY. Public parameters stay unchanged.
- ▶ Withdrawal: rotate the Controller's ABE master key and public parameters; remaining members obtain DKEYs for their remaining permissions.

The master key stays with the Controller. Rotation is Controller-wide. Participants must retrieve and validate updates before applying them; disconnected nodes cannot do so immediately.

Discovering Updates Does Not Make Revocation Instantaneous

1. Revised design: the Controller publishes new public parameters as signed Data; Sync announces availability.
2. Participants retrieve and validate parameters and permission status, then refresh DKEYs as required.
3. Grants also require status discovery, even when public parameters do not change.

Local status retrieval and rotation exist; dedicated Sync announcements still need end-to-end verification. Measure propagation, rejection delay and rekey traffic. Neither NDNSF nor a certificate-revocation alternative can recall disclosed keys or plaintext.

Threat Model: What Is Trusted?

- ▶ Trusted: Controller authorization and the User's planning decision.
- ▶ Untrusted delivery: an object may arrive through a producer, forwarder, cache, or repository and still requires validation.
- ▶ Adversarial cases to test: wrong identities, permissions, challenges, request state, roles, dependencies, or execution identities.
- ▶ Not established by signatures/ABE: honest resource reports, correct inference, truthful images, or resistance to authorized key sharing.

No Byzantine-coordinator consensus claim; no traffic-analysis or instantaneous-revocation guarantee.

RQ2 Extension: From Positive ACKs to a Role Plan

1. Application supplies roles, dependency template, constraints and deadline; the template may be handwritten.
2. The User closes the ACK window and freezes the eligible, validated positive ACKs.
3. The User applies a replaceable planning policy and signs the resulting role assignments and dependencies.
4. Each Provider validates the plan, derives permitted inputs and rechecks its resources before starting.

The snapshot freezes observed candidates for this decision, not every Provider in the network or their future availability.

A Plan Is More Than a Request ID

Existing request ID	Proposed task-specific plan rules
Identifies the request; requires a validated message	Assigns exact Providers to collection and processing roles
Does not alone specify predecessors	Declares which producer role may supply each named dependency
Does not alone describe key scope	Associates permitted objects, execution identity, and authorization/key scope

Example: within the same request, a service-authorized UAV that was not assigned “view 1” must not satisfy the view-1 dependency.

Plan-to-Data Enforcement: Check Before Advancing

1. Validate the Data signature using the appropriate trust rules.
2. Check active request, plan, execution identity and deadline.
3. Verify that the signer is the assigned predecessor Provider.
4. Check that the object matches the declared edge, permitted name, data format and key scope.
5. Require complete validated input before the dependent computation starts.

Apply the same rules to direct, cached and repository-fetched Data. This is a proposed acceptance contract; full end-to-end qualification remains required.

Collaboration Recovery Needs Explicit State

Event	Required behavior
Late ACK	Do not silently change the committed plan
Duplicate Selection	Check whether execution was already authorized; do not repeat a side effect
Incomplete dependency	Wait within policy/deadline, recover, or fail; do not execute on partial data
Provider replacement	Authorize a new assignment; reject old execution actions and specify permitted result reuse
Cancellation	Prevent late messages from reauthorizing cancelled work

Network re-retrieval is not task re-execution. Compensation is not an atomic transaction or exactly-once physical action.

NDNSF-UAV: Application-Level Validation

Unmanned aerial vehicle (UAV) application workflows exercise several framework mechanisms.
[28]

Application function	NDNSF validation scope
Control, telemetry, mission status	Provider/request association, permissions, operation lifecycle
Live video and recordings	Stream lifecycle, protected delivery, named-object retrieval
Detection and multi-view tasks	Service provision/consumption, role assignments, dependencies

Coverage is broader than one collaboration case. Each claim still needs evidence from the actual NDNSF path; flight safety and image-analysis accuracy need separate validation.

UAV Collaboration Example: Capture, Analyze and Return

An operator requests ground-area views from nearby UAVs, followed by analysis.

1. After Selection, assigned UAVs capture views and sign encrypted ViewImage Data under their authorized namespaces.
2. The compute Provider retrieves images over assigned edges and publishes Annotation Data.
3. Its encrypted result gives the requester original image names and permitted image keys.
4. The requester retrieves the original images and displays the annotations.

Original image Data remains under its producer's name. GPS alone does not establish camera visibility.

UAV Evaluation: Completion and Correct Input Use

- ▶ A complete authorized collection–processing–return task before the deadline.
- ▶ Correct handling of unavailable views, delayed images and a failed processing Provider.
- ▶ Reject wrong-role/edge inputs; accept cached images only when they satisfy the same task checks.
- ▶ MiniNDN [22] and PX4 SITL [23] results clearly distinguished from physical-flight evidence.

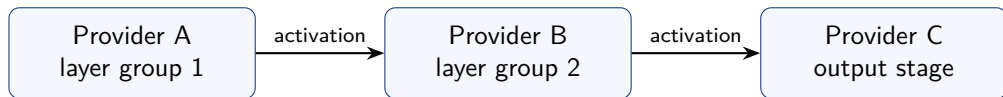
Higher recognition/localization accuracy is a separate claim requiring ground truth and a single-view control. It is not assumed here.

NDNSF-DI: Application-Level Validation

Application function	NDNSF validation scope
Single-Provider inference	Discovery, permissions, input and result delivery
Model preparation and reuse	Readiness, artifact references, named-object retrieval
Incremental output and conversation	Request state, cancellation, output lifecycle
Multi-Provider inference	Assignments, dependencies, recovery

The application also tests the invocation foundation, without requiring a model split. Interfaces and focused tests exist; complete model/device/recovery qualification remains open. [28]

DI Collaboration Example: Named Computation Dependencies



- ▶ Pipeline: sequential layers/groups; tensor partitioning additionally needs supported collectives.
- ▶ ONNX [25] describes the graph; export alone does not implement distributed execution.
- ▶ Repo [24] stores artifacts; forwarding caches do not guarantee persistence.

The framework contribution is checked runtime assignment and data exchange, not model partitioning itself.

DI State and Evidence Boundaries

- ▶ Model weights, reusable artifacts, request activations and KV state have different lifetimes.
- ▶ With KV caching, the runtime reuses prior key/value (K/V) tensors and supplies next-step inputs; resident weights do not replace this state. [31]
- ▶ An in-process test does not establish correct execution across processes or devices.
- ▶ Freeze one supported model/partition first; tensor/hybrid and LLM streaming need their own evidence if claimed.

Earlier small-model measurements do not validate the newer distributed inference and recovery paths. [28]

Application Coverage and Observable Outcomes

1. Name the application function and the production NDNSF API path it uses.
2. Specify the input, expected result, failure condition, and tested configuration.
3. Record evidence separately: implementation, focused test, multi-process test, measured outcome.

UAV and DI cover discovery, authorization, data delivery, lifecycle, and collaboration. Local calls do not prove remote behavior. Representative workflows cover claimed mechanisms; optional application features need not become separate dissertation requirements.

RQ3: Define the Baselines Before Comparing

- ▶ NSC [10]: named service calls; describe the tested executor, timeout, queue and recovery behavior.
- ▶ gRPC [17]: describe resolver, sequential/parallel policy, deadlines, retry and cancellation.
- ▶ Match Provider count, capacity, task, topology/traces, publication times and measurement windows.
- ▶ Report fixed-list and dynamically discovered configurations as different configurations.

An incomplete baseline cannot establish a limitation of the full framework.
Historical unmatched data remain diagnostics.

Mobility: Completion under Different Coverage Conditions

Ten seeds (62–71), four Providers/system, 2 m/s; mean seed completion rate. [28]

AP radius	NDNSF	gRPC	NSC
50 m	54.57%	53.80%	55.47%
100 m	97.97%	97.97%	98.33%
150 m	100.00%	100.00%	100.00%

No overall NDNSF advantage is established. The next slide uses a separate holdout campaign (seeds 72–81), not a subset of these runs.

Mobility: Lower Tail Latency in a Defined Subset

Seeds 72–81; four Providers/system; 100 m, 2 m/s; 1,312 aligned switching requests. [28]

Comparison	Seed-p95 difference	95% CI
NDNSF minus gRPC-SEQ-4	−597 ms	[−992, −200] ms
NDNSF minus NSC-4	−2618 ms	[−2916, −2315] ms

Completed: NDNSF 1,311; gRPC 1,297; NSC 1,296 (of 1,312 each).

Mean paired seed-p95 differences; each p95 uses that system's successful requests. Different failures limit the comparison; no general advantage is inferred.

Fewer Execution Starts than the Parallel Baseline

Six-seed, capacity-matched comparison; 1,800 requests per system. [28]

Configuration	Completed requests	Execution starts
NDNSF	1,798 / 1,800	1,800
gRPC-PAR-4	1,800 / 1,800	7,180

NSC-4 also completed 1,800/1,800, with 1.002 starts/request (rounded). No work-efficiency advantage over NSC is claimed.

Fewer starts than gRPC-PAR-4 is not proof of fewer bytes, less CPU time or lower energy. Similar observed completion is not proven equivalence.

A Useful Negative Result: Splitting Can Be Slower

Earlier small Qwen-style ONNX model; ten successful runs per layout. [28]

Layout	Mean latency	p95
Shared-backbone distributed layout	259.4 ms	283.9 ms
Single Provider	186.8 ms	206.3 ms

Do not split merely because several Providers exist. Communication/setup overhead is a candidate explanation, not a causal result established by this measurement.

Retained historical diagnostic; raw-run/model provenance remains to be rechecked. Not evidence for the newer distributed design.

Remaining Experiments Follow the Claims

RQ	Required evidence	Outcome and cost
RQ1	ABE-backed versus DNMP-inspired rules plus encryption; unauthorized call, replay, grant and withdrawal	No unauthorized handler start; discovery delay, key/policy size, status delay and rekey traffic
RQ2	One UAV and one DI graph; wrong role/edge; direct/cache parity; cancellation/replacement	Correct task completion; validation, transfer and recovery cost
RQ3	Matched workload and failure schedules; feasible single-device control; cold/warm state	Completion plus latency, redundant work, bytes, uncertainty, and cases with no advantage

Planned tests are not completed findings. Record all scheduled runs and failure boundaries.

A Bounded Path to Spring 2027

Internal target	Required deliverable
September 2026	Agree scope/RQs; freeze one UAV graph and one DI partition; resolve correctness blockers
October–November 2026	Complete registered experiments; freeze evidence and write bounded RQ answers
December 2026	Complete integrated advisor-review draft
January 2027	Committee review and readiness decision; reduce scope or revise timing if core evidence is missing
February 2027	Target defense, conditional on evidence and committee approval

Committee draft: two weeks before defense. Spring 2027 application/candidacy: February 5; final ProQuest: March 26. Recheck before filing. [29,30]

Expected Research Outcomes

- ▶ **RQ1 answer:** when ABE-backed authorization is preferable to DNMP-inspired role rules plus encryption, accounting for confidentiality, management and withdrawal costs.
- ▶ **RQ2 answer:** exactly which task-assignment mistakes are rejected and which recovery behavior is supported.
- ▶ **RQ3 answer:** measured benefits, negative results and deployment limits.
- ▶ Reproducible configurations and evidence linking each conclusion to its tested scope.

The outcome is a tested service protocol and an evidence-backed account of when its mechanisms are worth their cost.

Discussion: Is This Scope Sufficient and Feasible?

1. Are the three research questions specific enough to support defensible answers?
2. Are the authorization alternatives complete and fairly configured?
3. Are one UAV graph and one supported DI partition sufficient to test the abstraction?
4. Which optional features should be excluded if they delay the core evidence?

NDNSF: evaluate runtime discovery and authorized invocation; extend the foundation to role-aware collaboration.

Backup: Discovery Configuration Is Not Framework Capability

Provider-transition configuration	Completed
NDNSF, no pre-registered Provider addresses	357 / 357
Static-list NSC/gRPC configurations	0 / 357
Pre-registered baseline variants	357 / 357

This isolates the configured discovery boundary. Resolver/discovery integration can update candidates; it is not proof that gRPC or NSC cannot do so. [27,28]

Backup: Historical Loss Evidence Is Not a New Matched Test

Historical per-link loss	NDNSF completed requests
1%	599 / 600
3%	598 / 600
10% (best recorded run)	539 / 600

A selected run is not an aggregate estimate. Do not combine these numbers with another batch and label the result “matched.” [28]

Per-link packet loss is not the probability of service failure: retrieval retries and the request deadline also affect completion.

References: NDN and Authorization

- [1] L. Zhang et al., “Named Data Networking,” ACM SIGCOMM CCR, 2014.
- [2] T. Li et al., “A Brief Introduction to NDN Dataset Synchronization (NDN Sync),” MILCOM, 2018.
- [3] P. Moll et al., “A Brief Introduction to State Vector Sync,” NDN-0073, rev. 2, 2021.
- [4] Z. Zhang et al., “NAC: Automating Access Control via Named Data,” MILCOM, 2018.
- [12] K. Nichols, “Lessons Learned Building a Secure Network Measurement Framework using Basic NDN,” ACM ICN, 2019.
- [16] Y. Yu et al., “Schematizing Trust in Named Data Networking,” ACM ICN, 2015.
- [19] S. Dulal et al., “Enhancing NAC-ABE to Support Access Control for mHealth Applications and Beyond,” arXiv:2311.07299, 2023.
- [26] NDN Project, “Data Packet,” NDN Packet Format Specification.
docs.named-data.net/NDN-packet-spec/current/data.html (accessed Sept. 2026).

References: Invocation and Composition

- [6] C. Tschudin and M. Sifalakis, “Named Functions and Cached Computations,” IEEE CCNC, 2014.
- [7] M. Krol and I. Psaras, “NFaaS: Named Function as a Service,” ACM ICN, 2017.
- [9] M. Krol et al., “RICE: Remote Method Invocation in ICN,” ACM ICN, 2018.
- [10] D. Meirovitch and L. Zhang, “NSC—Named Service Calls, or a Remote Procedure Call for NDN,” NDN-0074, rev. 1, 2021.
- [15] C. Barrios, E. Alker and M. Kumar, “CaSCON: Cache-Assisted Service Composition Orchestration Leveraging NDN,” IEEE SMARTCOMP, pp. 122–129, 2025. DOI: 10.1109/SMARTCOMP65954.2025.00061.
- [17] gRPC Authors, “Core Concepts, Architecture and Lifecycle,” gRPC documentation.
- [27] gRPC Authors, “Custom Name Resolution,” grpc.io/docs/guides/custom-name-resolution/ (accessed Sept. 2026).

References: Tools, Evidence and Degree Planning

- [22] Mini-NDN, project documentation: minindn.ndn.today.
- [23] PX4, "Software-in-the-Loop Simulation," PX4 User Guide.
- [24] NDN Project, "Repo Protocol Specification," [repo-ng](https://repo-ng.com) documentation.
- [25] ONNX, "Open Neural Network Exchange Intermediate Representation Specification."
- [28] NDN SF, research-revision-audit.md, companion claim/evidence ledger, September 9, 2026. Contains campaign paths and required validation.
- [29] University of Memphis CS, "Information for Computer Science Ph.D. Students," [forms_grad_phd_programinfo.pdf](#) (accessed September 9, 2026).
- [30] University of Memphis Graduate School, "Graduate School Calendar," Spring 2027. memphis.edu/gradschool/news-events/calendar.php (checked September 14, 2026).
- [31] ONNX Runtime, "How to configure the past present share buffer," GenAI documentation (accessed September 11, 2026).